

```

import components.naturalnumber.NaturalNumber;
import components.naturalnumber.NaturalNumberSecondary;

/**
 * {@code NaturalNumber} represented as a {@code String} with implementations of
 * primary methods.
 *
 * @convention <pre>
 * [all characters of $this.rep are '0' through '9'] and
 * [$this.rep does not start with '0']
 * </pre>
 * @correspondence <pre>
 * this = [if $this.rep = "" then 0
 *         else the decimal number whose ordinary depiction is $this.rep]
 * </pre>
 *
 * @author cl_c2231am03 (Tyler zhuangzhuang He & Meng Zhang)
 */
public class NaturalNumber3 extends NaturalNumberSecondary {

    /**
     * Private members -----
     */

    /**
     * Representation of {@code this}.
     */
    private String rep;

    /**
     * Creator of initial representation.
     */
    private void createNewRep() {
        this.rep = "";
    }

    /**
     * Constructors -----
     */

    /**
     * No-argument constructor.
     */
    public NaturalNumber3() {
        this.createNewRep();
    }

    /**
     * Constructor from {@code int}.
     *
     * @param i
     *         {@code int} to initialize from
     */
    public NaturalNumber3(int i) {
        assert i >= 0 : "Violation of: i >= 0";

        this.createNewRep();
        if (i > 0) {
            this.rep = Integer.toString(i);
        }
    }

    /**

```

```

    * Constructor from {@code String}.
    *
    * @param s
    *       {@code String} to initialize from
    */
    public NaturalNumber3(String s) {
        assert s != null : "Violation of: s is not null";
        assert s.matches("0|[1-9]\\d*") : ""
            + "Violation of: there exists n: NATURAL (s = TO_STRING(n))";

        this.createNewRep();
        if (!s.equals("0")) {
            this.rep = s;
        }
    }

    /**
     * Constructor from {@code NaturalNumber}.
     *
     * @param n
     *       {@code NaturalNumber} to initialize from
     */
    public NaturalNumber3(NaturalNumber n) {
        assert n != null : "Violation of: n is not null";

        this.createNewRep();
        if (!n.isZero()) {
            this.rep = n.toString();
        }
    }

    /*
     * Standard methods -----
     */

    @Override
    public final NaturalNumber newInstance() {
        try {
            return this.getClass().getConstructor().newInstance();
        } catch (ReflectiveOperationException e) {
            throw new AssertionError(
                "Cannot construct object of type " + this.getClass());
        }
    }

    @Override
    public final void clear() {
        this.createNewRep();
    }

    @Override
    public final void transferFrom(NaturalNumber source) {
        assert source != null : "Violation of: source is not null";
        assert source != this : "Violation of: source is not this";
        assert source instanceof NaturalNumber3 : ""
            + "Violation of: source is of dynamic type NaturalNumber3";
        /*
         * This cast cannot fail since the assert above would have stopped
         * execution in that case.
         */
        NaturalNumber3 localSource = (NaturalNumber3) source;
        this.rep = localSource.rep;
        localSource.createNewRep();
    }

    /*

```

```
* Kernel methods -----
*/

@Override
public final void multiplyBy10(int k) {
    assert 0 <= k : "Violation of: 0 <= k";
    assert k < RADIX : "Violation of: k < 10";
    if (k != 0 || !this.isZero()) {
        this.rep = this.rep + k;
    }
}

@Override
public final int divideBy10() {
    int result = 0;
    if (!this.isZero()) {
        int lastIndex = this.rep.length() - 1;
        result = Character.digit(this.rep.charAt(lastIndex), RADIX);
        this.rep = this.rep.substring(0, lastIndex);
    }
    return result;
}

@Override
public final boolean isZero() {
    return this.rep.equals("");
}
}
```